Artificial Intelligence 2025/2026

Assignment: Adversarial Search Strategies and Decision Trees

Deadline Submission: May 17, 2026

1 Introduction

The material related to this assignment can be found in the books and slides suggested for the course. Besides, quite a lot of material is available about adversarial games and decision trees on the internet (beware to choose wisely).

This work is to be submitted via *Moodle*. It will be developed partially during practical classes, but students are expected to complement it with extra-class work.

The students must be organised into groups of 3.

In this assignment, you will learn to design and implement a relatively simple program that is capable of playing *PopOut* with a human.

2 PopOut: The Game

PopOut is a variant of Connect-4. It starts the same as traditional gameplay, with an empty board and players alternating turns placing their own coloured discs into the board. During each turn, a player can either add another disc from the top, or if one has any discs of their own colour on the bottom row, remove (or "pop out") a disc of one's own colour from the bottom. Popping a disc out from the bottom drops every disc above it down one space, changing their relationship with the rest of the board and changing the possibilities for a connection. The first player to connect four of their discs horizontally, vertically, or diagonally wins the game. For example, consider the state of the game shown in Figure 1[1].

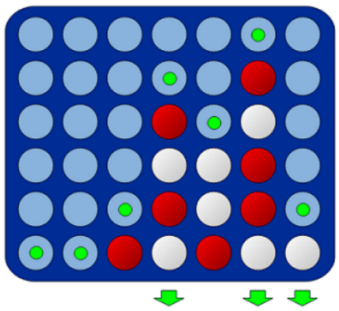

Figure 1: It is White's turn to move and it can choose among 7 drop moves (green dots) and 3 pop moves (green arrows).

PopOut differs from Connect-4 by the introduction of pop moves and three additional rules to handle draws and wins[2]. These three rules are needed to remove ambiguity about what the result of the game is:

[1] https://oulurepo.oulu.fi/handle/10024/39813

[2] Allen, J. D. (2010). The complete book of Connect-4: History, strategy, puzzles. New York, NY: Sterling.

- The first rule deals with simultaneous four-in-rows. If a pop move creates four-in-rows for both players, the player who made the pop move is declared a winner and the other player's four-in-row is ignored.
- The second rule allows the game to be drawn if the board is full. In such a case, the player to move decides whether he wants to make a pop move or end the game as a draw.
- The third rule handles repetitions. If the same state is repeated three times, either player can declare the game drawn.

3 Game Interface

The interface for the game can be something like the one shown in Figure 2.

```
-------
-------
-------
-------
X--O-XO
X-OXOXO

It is now X's turn.
Make a move by choosing your coordinates to play and the type of move (drop or pop).
```

Figure 2: Example of interface for PopOut

After X makes a move, the computer takes this new board and uses one of the two algorithms to be implemented. When the computer finishes choosing the best move among the possible ones, it will then exhibit a new board with the computer's move, and wait for the human to play.

Your implementation will need to support the following three game scenarios:

1. human vs. human
2. human vs. computer
3. computer vs. computer (2 different algorithms)

4 Work to develop

The goal of this assignment is to implement the Monte Carlo Tree Search (*MCTS*) algorithms and a decision tree for each given dataset learned using the *ID3* procedure.

4.1 Monte Carlo Tree Search

MCTS is an adversarial search algorithm; therefore, it takes into account that an adversary will change the system state in subsequent steps.

For example, given the configuration of Figure 1, it is white's turn to move and can choose among 7 drop moves (green dots) and 3 pop moves (green arrows), and your program needs to decide which is best.

You should implement the *MCTS* algorithm using the Upper Confidence Bound for Trees (UCT) as the evaluation function for each branch. In addition, you should analyse/explore different numbers of selected children for each node, and other strategies, not only keeping the standard implementation of *MCTS*.

4.2 Decision Trees

The goal of this part of the assignment is to write a program that learns a decision tree from a given training dataset using the *ID3* procedure.

In addition to learning a decision tree, your program must also be prepared to accept test examples, i.e., after generating your tree, you must be able to apply your tree to new examples and be able to classify them appropriately.

Important note: You are not allowed to use `scikit-learn` or other libraries to automatically define and train the decision trees. External libraries will be allowed only to manage and pre-process input data.

4.2.1 Datasets

In this assignment, you will be required to consider 2 different datasets:

1. The first dataset (available on the *Moodle* platform) can be considered a "warm-up" for testing your decision tree implementation. The first dataset is the **iris dataset**: it contains **numerical** information about plants of three classes (iris setosa, iris virginica and iris versicolor). The attributes are petal length and width, and sepal length and width. The task is to learn a decision tree that can decide to which class a plant belongs to, given these attributes. It is supposed to present the output (tree) visually for all these datasets.

   Important note: The dataset iris contains numerical values. You need to implement a way of discretising these values in order to minimize the size of your decision tree.

2. The second dataset is directly related to the game of *PopOut*, and it is not available on *Moodle*. You will need to generate it, in particular, using your previous implementation of the *MCTS* algorithm. You will need to generate a dataset of pairs $(state_i, move_i)$ where $state_i$ refers to the current state of the game and $move_i$ is the corresponding next move suggested by the algorithm. Using this dataset, you then need to train a decision tree using the *ID3* procedure that will be able to decide the next move given the current game state.

4.3 Submission of the solution

The solution should be delivered in *Moodle* by May 17, 2026, at 23:59:59, Lisbon time.

You should submit the following materials:

- Final code solution, as a notebook;
  - you should document your notebook, explaining your decisions and discussing the results obtained using the different alternatives tested;
- Slides for presentation (PDF format); This is a very short and to-the-point presentation (maximum of 10 minutes), summarising the following:
  - the problem;
  - your solution;
  - the results.
- Filled auto-evaluation file provided by the Professors.

4.4 Presentation of the solution

Students must present their work during the practical class in the last 2 weeks, as presented in the planning for the AI course. *For the presentation, students will use the slides included in the work submission.* During the presentation, students will be asked questions and may need to show the notebook submitted.

All group members must be present during the presentation. **If a member of the group is not present to the work presentation, he will receive a zero grade for this work, thus implying failing to pass.**

Each member of the group must comment on their contribution to the work, and must know what the other members of the group have done. **Failing to describe in detail what the presented solution is doing and why will determine a penalization in the overall evaluation of the project.**

4.5 Evaluation Criteria

Your work will be evaluated on the following criteria:

- 30% for the adversarial strategy implementation;
- 30% for the decision trees implementation;
- 30% technical skills: overall technical evaluation of the solution from a data science point-of-view, including rigour in the performance evaluation of the proposed solution;
- 10% soft-skills: essentially - your communication skills;

4.6 Classification

This assignment represents 30% of the grade for the course (6 values).

4.7 Some Tips

Be creative in your solution! Think of how you can use certain approaches in an unusual way, for example.

- Consider implementation constraints: understand the challenge well and identify any specific constraints regarding this challenge;
- Mention the constraints you are considering for the solution in the notebook;
- Work as a team: the time is very short, so we suggest that you distribute tasks well;